Comparison between NLP cleaning configurations

Disaster Tweet Clasification - Kaggle

Alessa Melo

Universidad Yachay Tech

School of Mathematical and Computational Sciences

March, 2026

Outline

1. Objective
2. Models Used
3. Exploratory Data Analysis (EDA)
4. Data Preprocessing
5. Performance comparison between models
6. Analysis

PROBLEM DESCRIPTION

Disaster Tweets Classification Dataset

Dataset Overview

- **Dataset:** Disaster Tweets Dataset (binary classification).
- **Goal:** Automatically detect tweets describing real disaster events.
- **Samples:** 7,613 tweets.

Dataset Composition

- **Not Disaster (Class 0):**
  - 4,342 tweets
  - 57.0% of the dataset
- **Disaster (Class 1):**
  - 3,271 tweets
  - 43.0% of the dataset

Disaster Tweet Classification Dataset

Modeling Challenges

- Tweets are short and often contain informal language, abbreviations, and slang.
- Text data must be converted into numerical representations before being used by machine learning models.
- Tweets frequently include noise such as URLs, mentions, hashtags, and emojis that require preprocessing.
- Some tweets use figurative language (e.g., “this exam is a disaster”), which makes classification more challenging.

Source: Kaggle Dataset — `https://www.kaggle.com/c/nlp-getting-started`

Objective

Project Objective

- Analyze how different **text preprocessing configurations** influence the performance of a **Multi-Layer Perceptron (MLP)** for disaster tweet classification.
- Train models using the **same neural network architecture, optimizer, loss function, batch size, and number of epochs** to ensure a fair comparison.
- Evaluate model performance using **Accuracy, Precision, Recall, F1-score, AUC, and Confusion Matrix**.

Methodology

Project Pipeline

1. **Exploratory Data Analysis (EDA)**
   - Dataset visualization and class distribution analysis
2. **Data Preprocessing**
   - **Configuration 1:** Lowercase, removal of URLs, mentions, and punctuation
   - **Configuration 2:** Configuration 1 + stopword removal
   - **Configuration 3:** Configuration 1 + lemmatization
3. **Model Training**
   - Fully Connected Neural Network (Multi-Layer Perceptron)
   - Same architecture used for all configurations
4. **Evaluation and Comparison**
   - Metrics: Accuracy, Precision, Recall, F1-score, and AUC
   - Confusion Matrix analysis
   - Comparative visualization of results

MODEL USED

Model

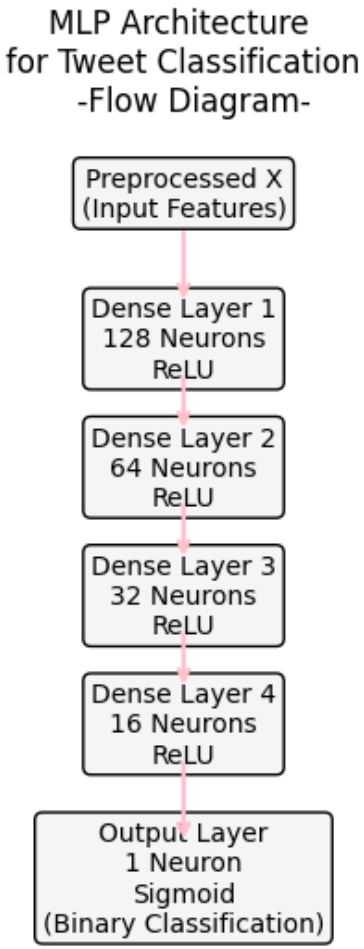


Figure: MLP Architecture

Model

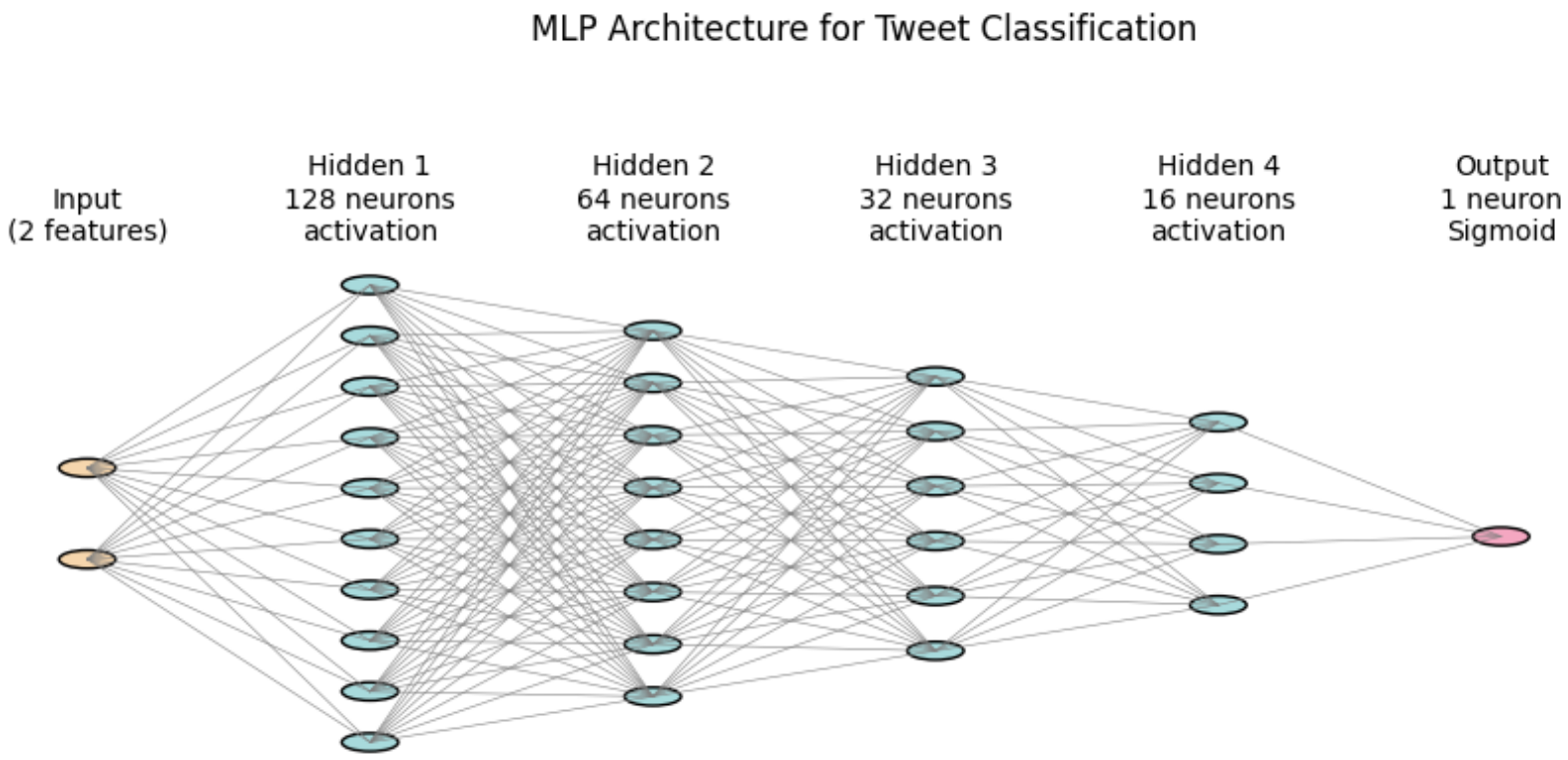


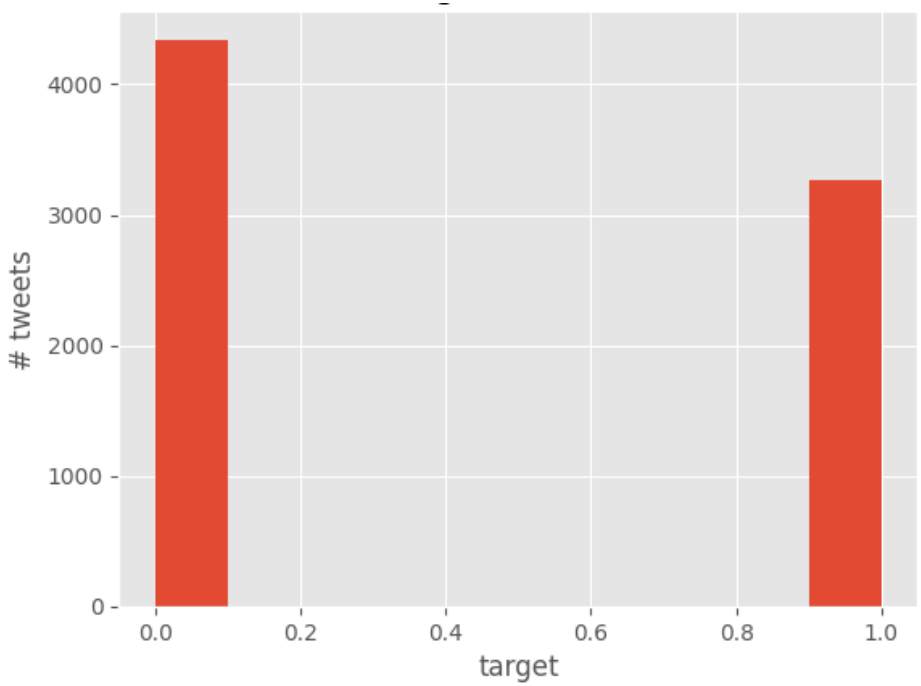
Figure: MLP Architecture for Binary Classification

EDA

Exploratory Data Analysis (EDA)

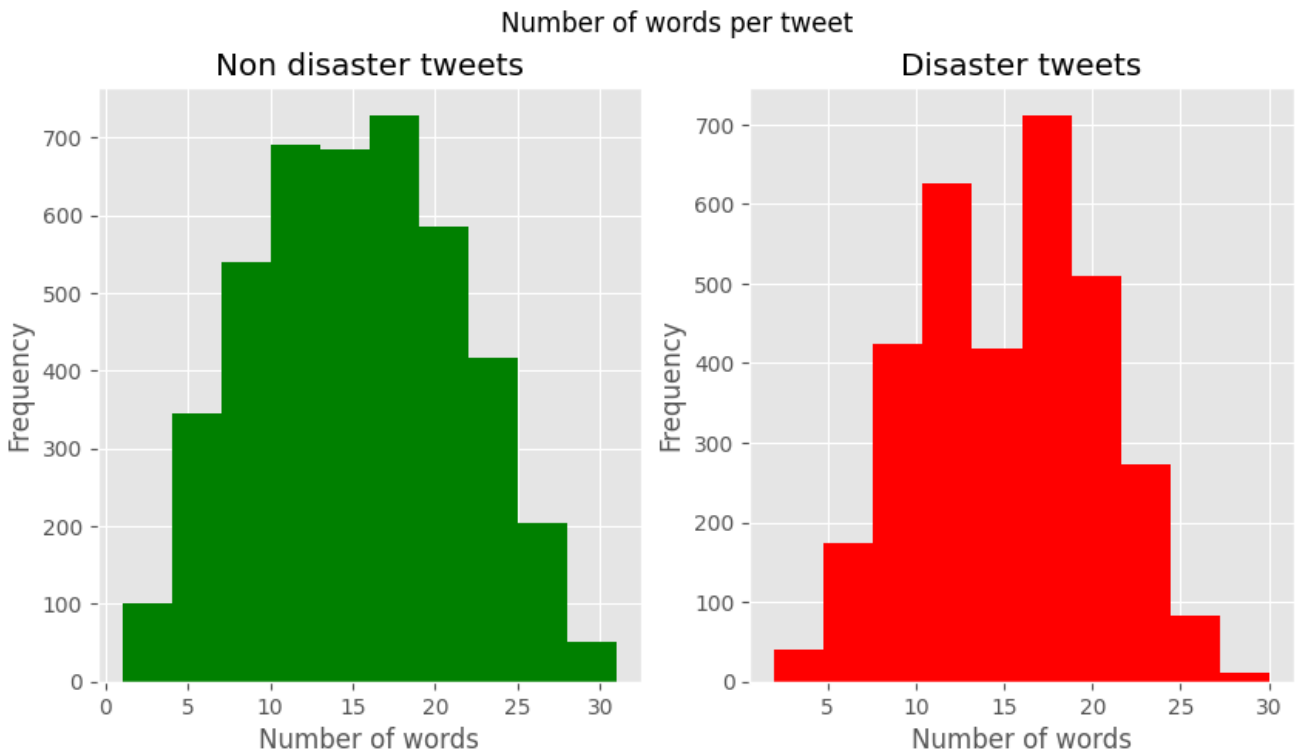
The distribution of the target variable shows a **moderate class imbalance**. The dataset contains slightly more tweets labeled as **non-disaster (Class 0)** than **disaster-related tweets (Class 1)**.

- Class 0 (Not Disaster): 4,342 tweets $\approx$ 57%
- Class 1 (Disaster): 3,271 tweets $\approx$ 43%



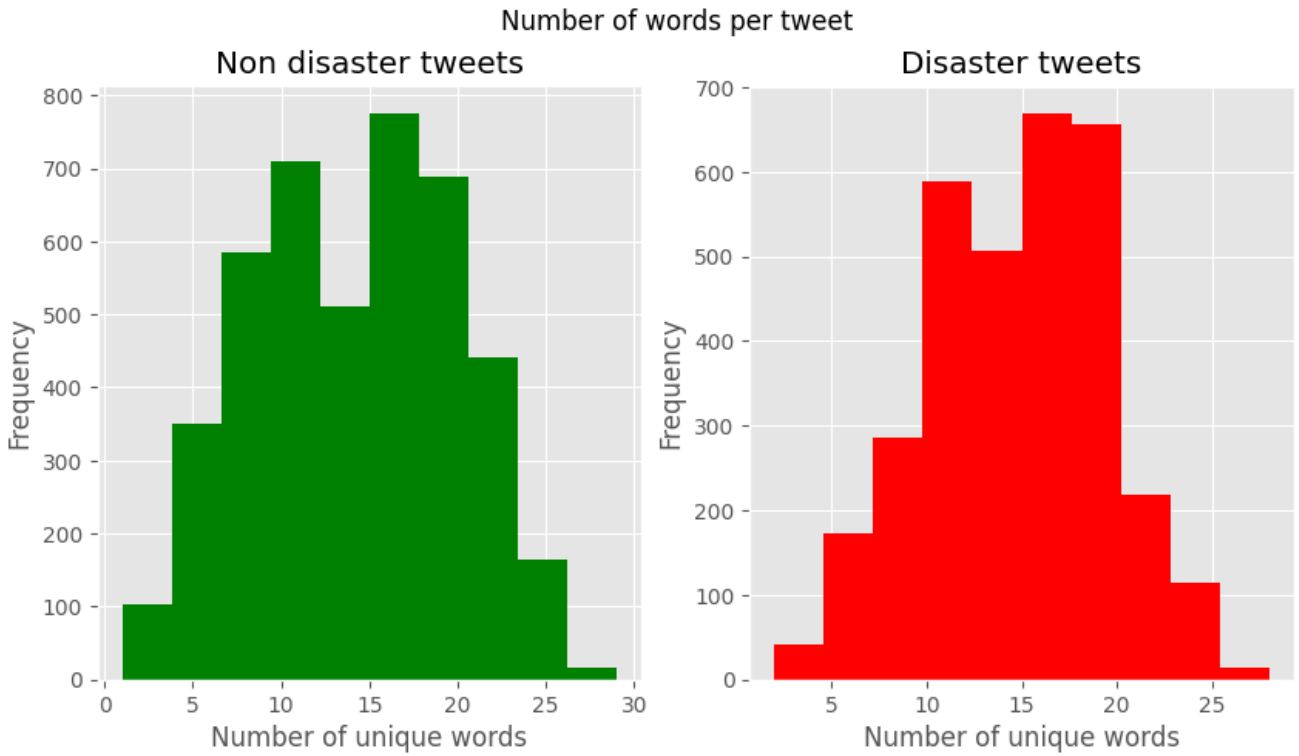
EDA: Word Count Distribution in Tweets

- Non-disaster tweets and disaster tweets exhibit a **similar distribution of word counts**.
- Most tweets contain between **10 and 20 words**, which reflects the short and concise nature of messages on social media platforms.



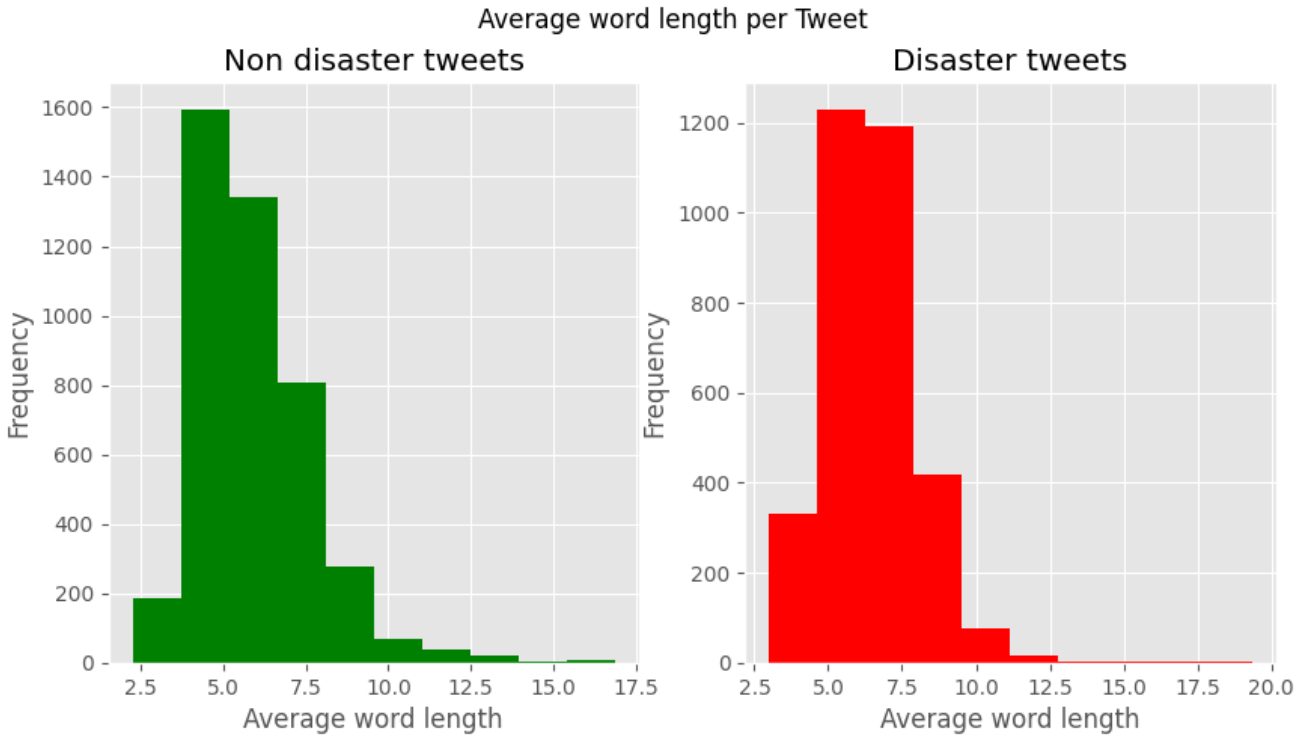
EDA: Unique Word Count Distribution

- Most tweets contain between **10 and 20 unique words**.
- The distributions for disaster and non-disaster tweets are **very similar**, suggesting that vocabulary diversity alone is not a strong indicator of disaster-related content.



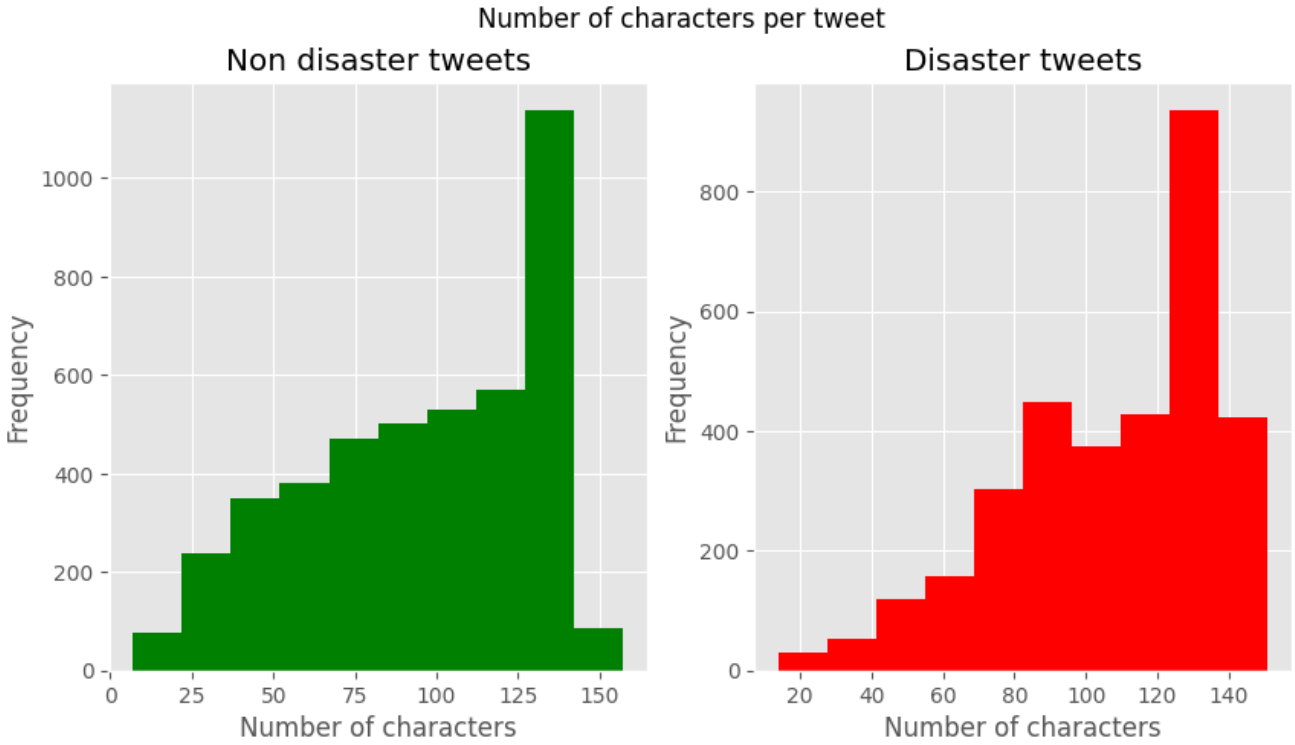
EDA: Average Word Length per Tweet

- Most tweets have an average word length between **4 and 7 characters**.
- The distributions of both classes are **very similar**, indicating that the average word length alone does not provide strong discriminative power for distinguishing disaster-related tweets.



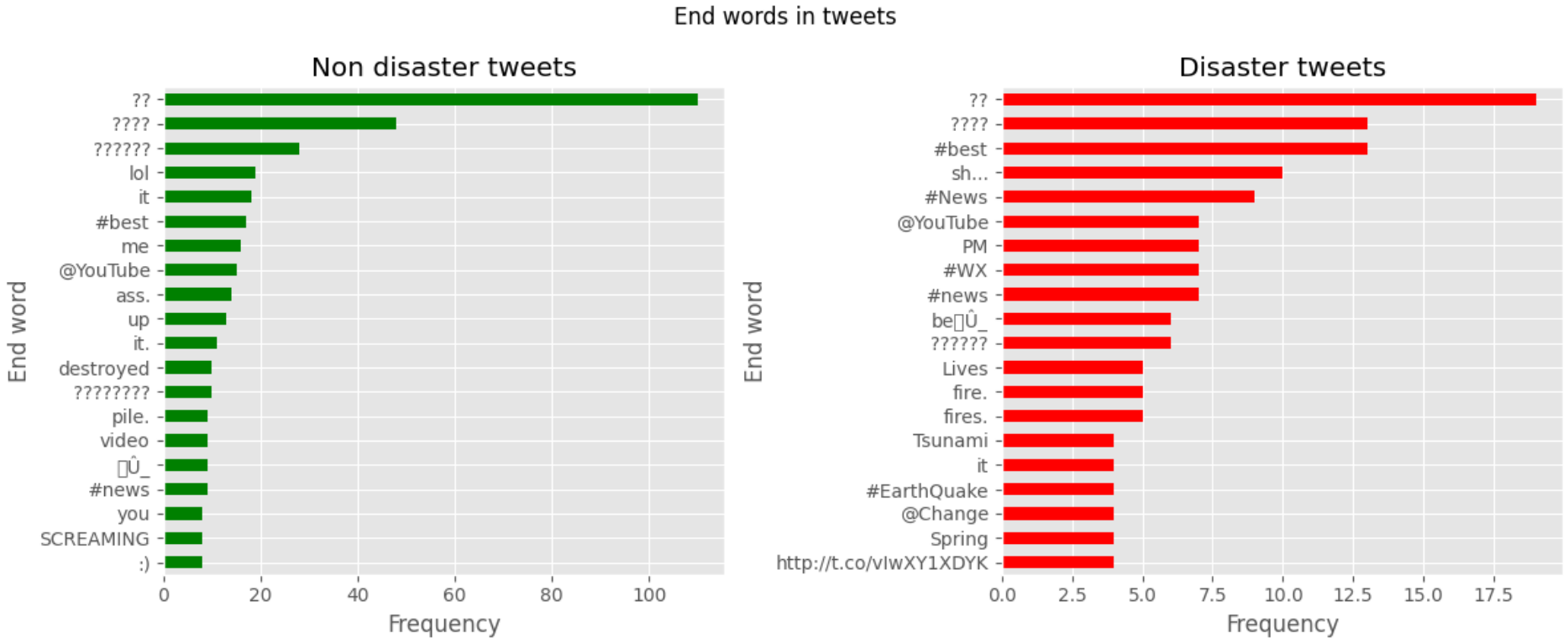
EDA: Number of characters per tweet

Most tweets contain between **80 and 140 characters**.
Both classes show a **similar distribution**, reflecting the typical short format of tweets on social media.

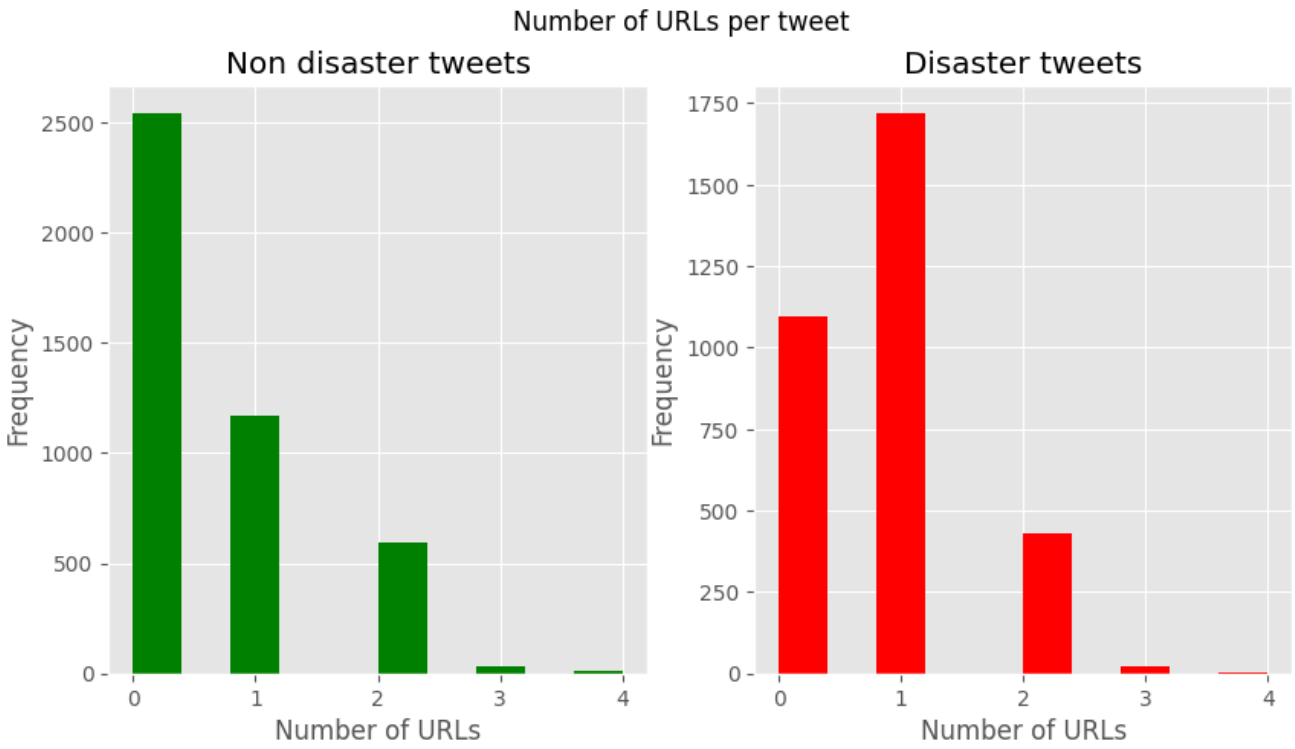


EDA: End Word

Non-disaster tweets frequently end with informal expressions such as *lol*, *it*, or conversational terms.
Disaster tweets tend to end with **keywords related to events**, such as *fire*, *news*, or references to media sources.
This indicates that the **context and vocabulary used in tweets** may contain important signals for classification.



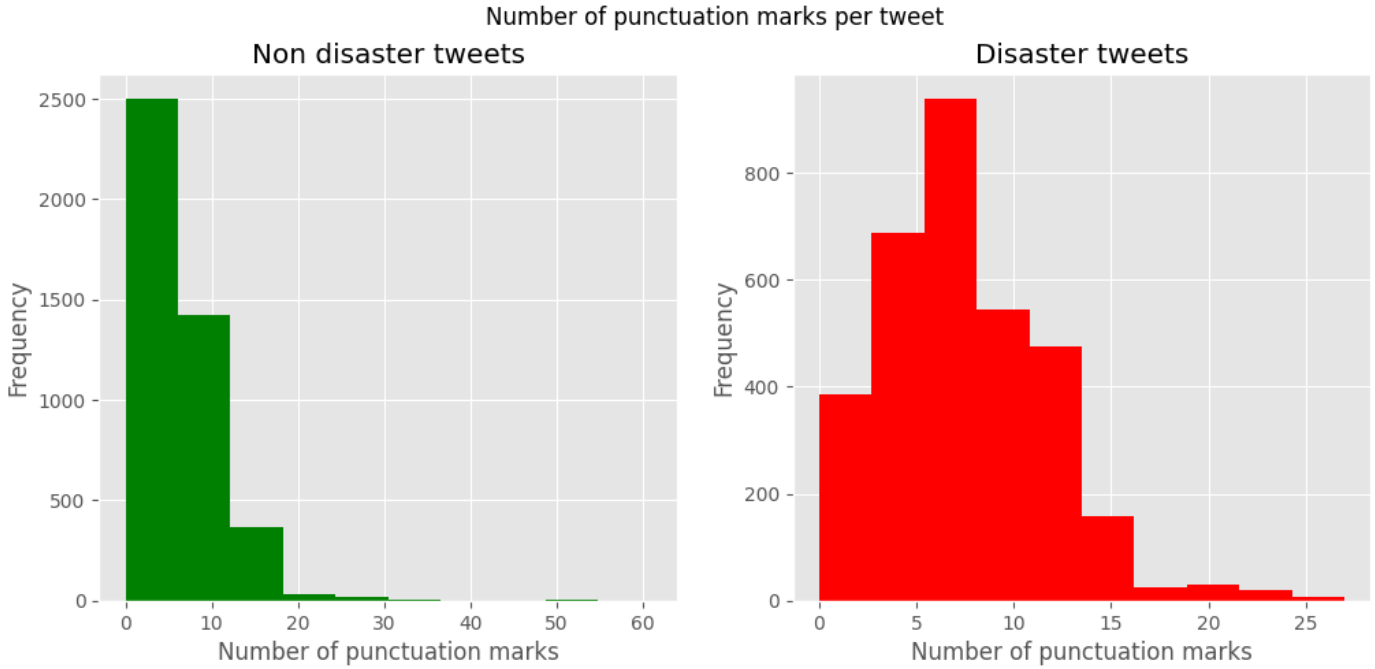
EDA: Number of URLs per tweet



Disaster-related tweets tend to include **more URLs**, often linking to news articles, reports, or external information about the event.
Non-disaster tweets are more likely to contain **no URLs**, reflecting casual conversation or personal posts.
The presence of URLs may therefore provide **useful contextual signals** for disaster-related content.

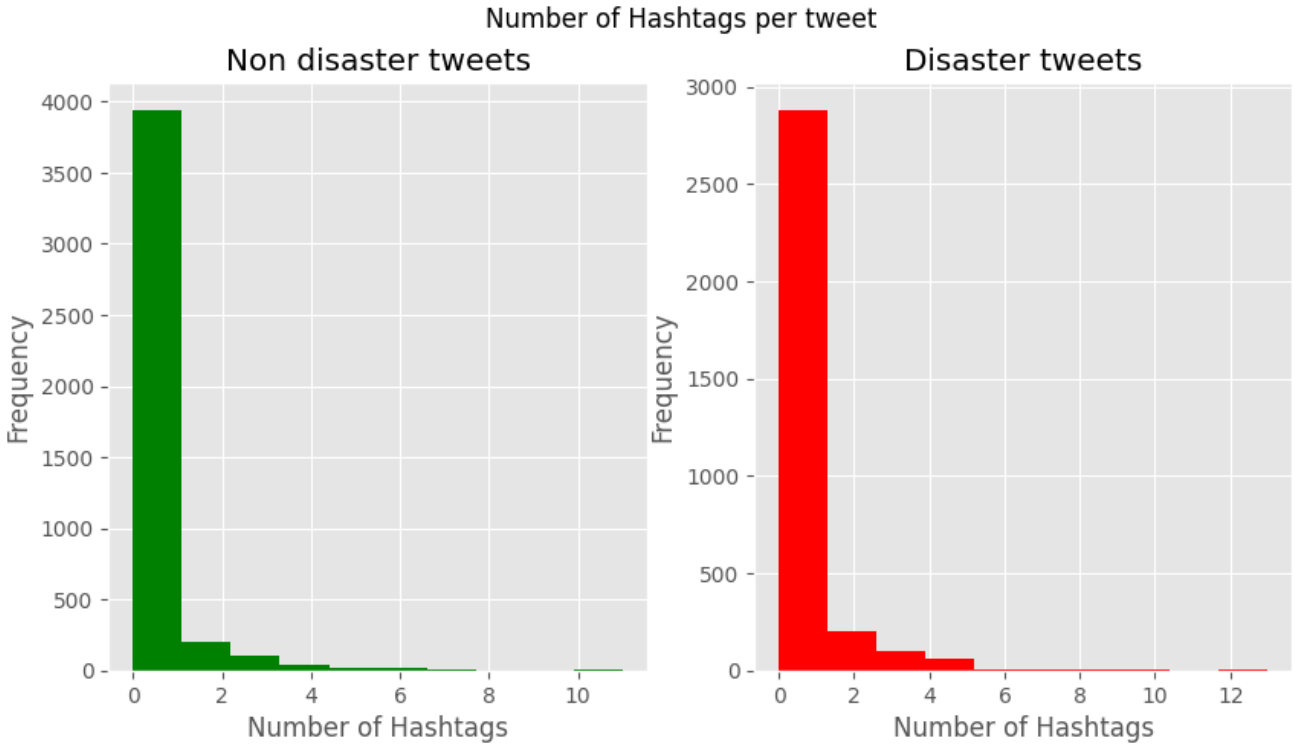
EDA: Number of punctuation marks per tweet

Disaster tweets tend to show **slightly higher punctuation usage**, possibly due to urgency or emphasis when describing events. Non-disaster tweets typically contain **fewer punctuation marks** and follow a more conversational style.



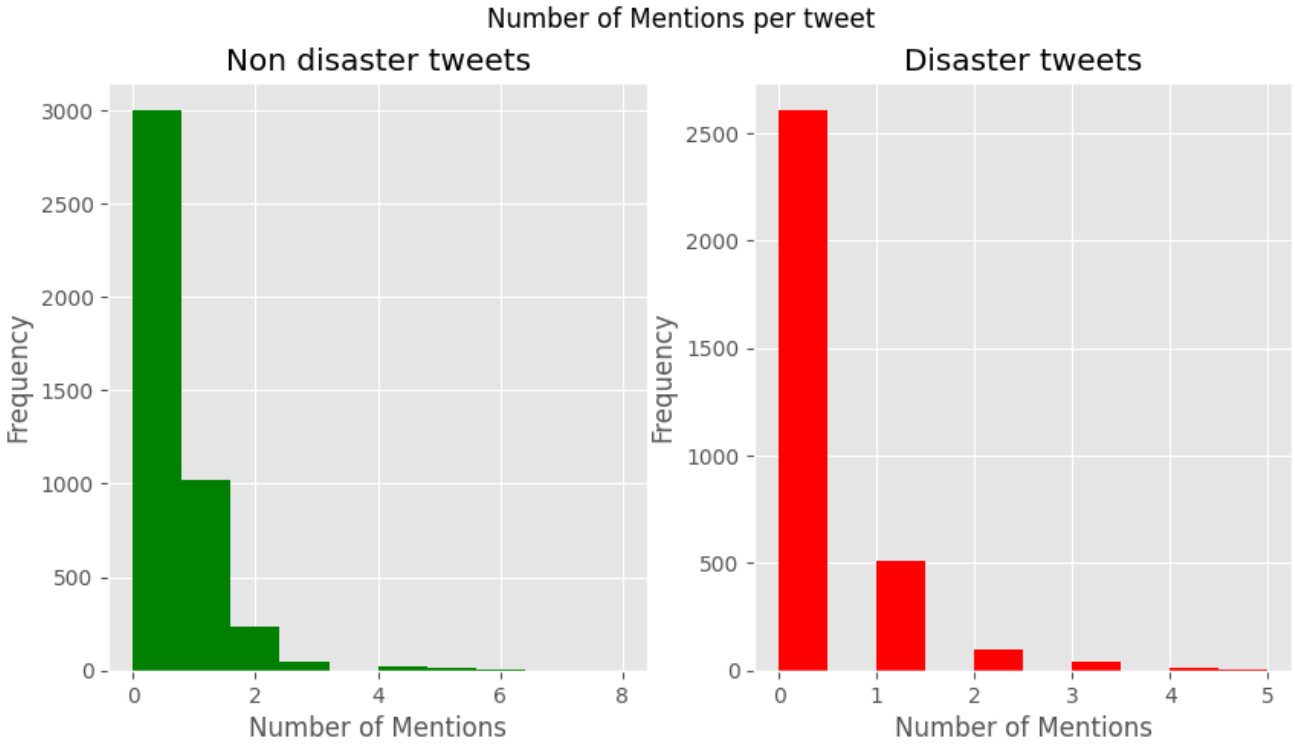
EDA: Number of hashtags per tweet

The majority of tweets contain **zero hashtags**.



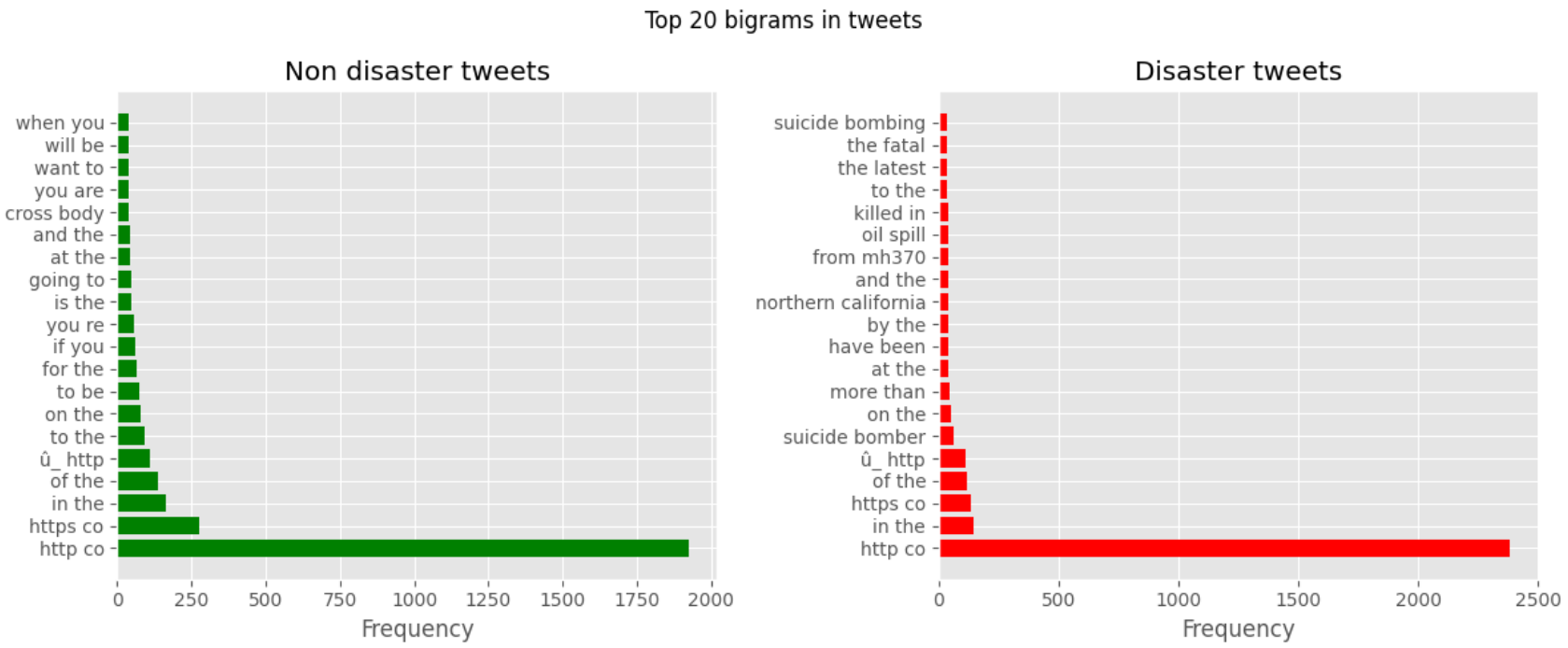
EDA: Number of mentions per tweet

Most tweets contain **zero mentions**.
Non-disaster tweets occasionally include **one or two mentions**, often used in conversational interactions.



EDA: Bigrams

Disaster tweets contain bigrams related to **events and emergencies**, such as words associated with fires, explosions, or accidents. Non-disaster tweets tend to include more **conversational phrases** or general expressions.



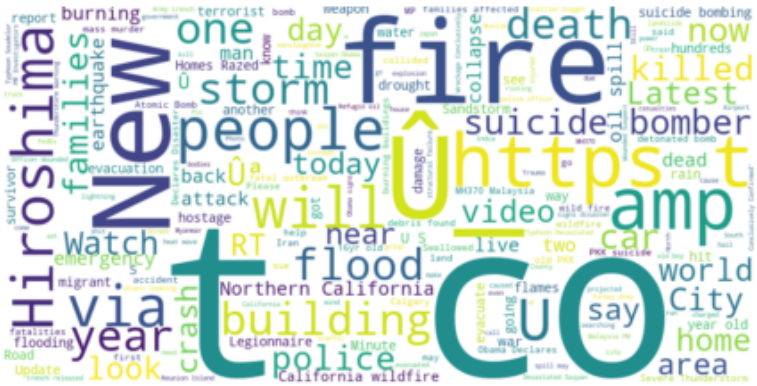
EDA: Wordclouds

Non-disaster tweets contain more **casual or conversational terms** commonly used in everyday communication.
Disaster tweets show a higher presence of **event-related words** such as *fire*, *storm*, *death*, or *police*.

WordCloud of Tweets

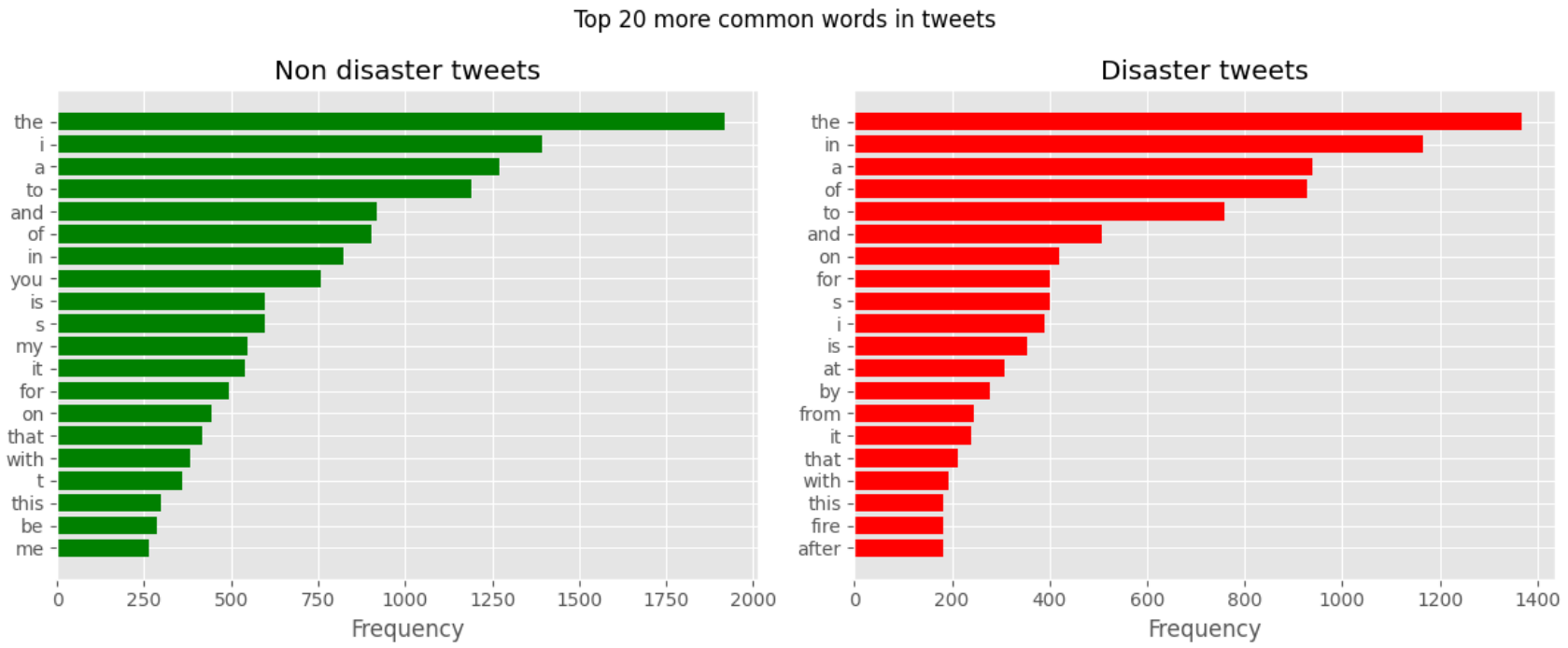
Non disaster tweets

Disaster tweets



Top Common Words

Many of the most frequent words are **stopwords** such as *the*, *to*, *and*, and *in*.
These words appear frequently in both classes and usually provide **limited semantic information**.
This observation supports the use of **text preprocessing** techniques such as stopword removal and normalization.



DATA PREPROCESSING

Data Preprocessing

Preprocessing Steps

Handling Missing Values: The original dataset includes the `keyword` and `location` columns. These features were explored during preprocessing, but following the assignment guidelines, the final model uses only the **tweet text** as input.

Initial Cleaning: Removal of HTML tags and emojis

Text Cleaning Configurations

- **Configuration 1:** Lowercase text, removal of URLs, mentions, and punctuation.
- **Configuration 2:** Configuration 1 + stopword removal.
- **Configuration 3:** Configuration 1 + lemmatization.

Lemmatization was chosen instead of stemming in order to preserve **valid dictionary words**. Each configuration generates a cleaned dataset that is saved as a `.csv` file and later used for model training.

Train–Validation–Test Split

Dataset Split and Training Setup

- **Train–Validation–Test Split:** The dataset was divided into training, validation, and test sets.
- **No Resampling Applied:** The original class distribution was preserved to maintain the dataset's natural imbalance.
- **Consistent Model Architecture:** All preprocessing configurations were trained using the **same neural network architecture** to ensure a fair comparison.
- **TF–IDF Representation:** Tweets were converted into numerical vectors using TF-IDF before being fed into the neural network.

PERFORMANCE

Overall Performance

The three preprocessing configurations achieve **very similar performance**. However, to better visualize the small differences between them, the following plot uses a **logarithmic scale**.

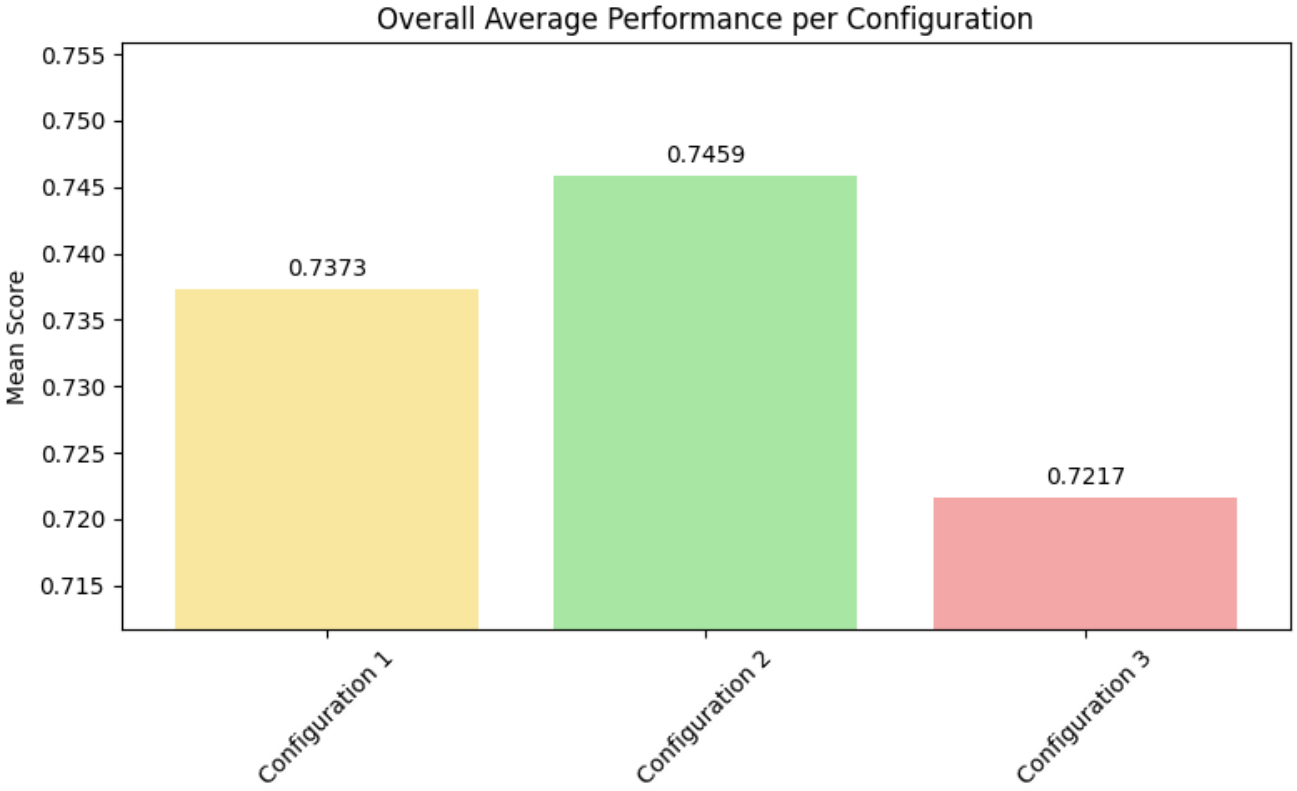


Figure: Comparison of the average performance across preprocessing configurations.

Configuration 1

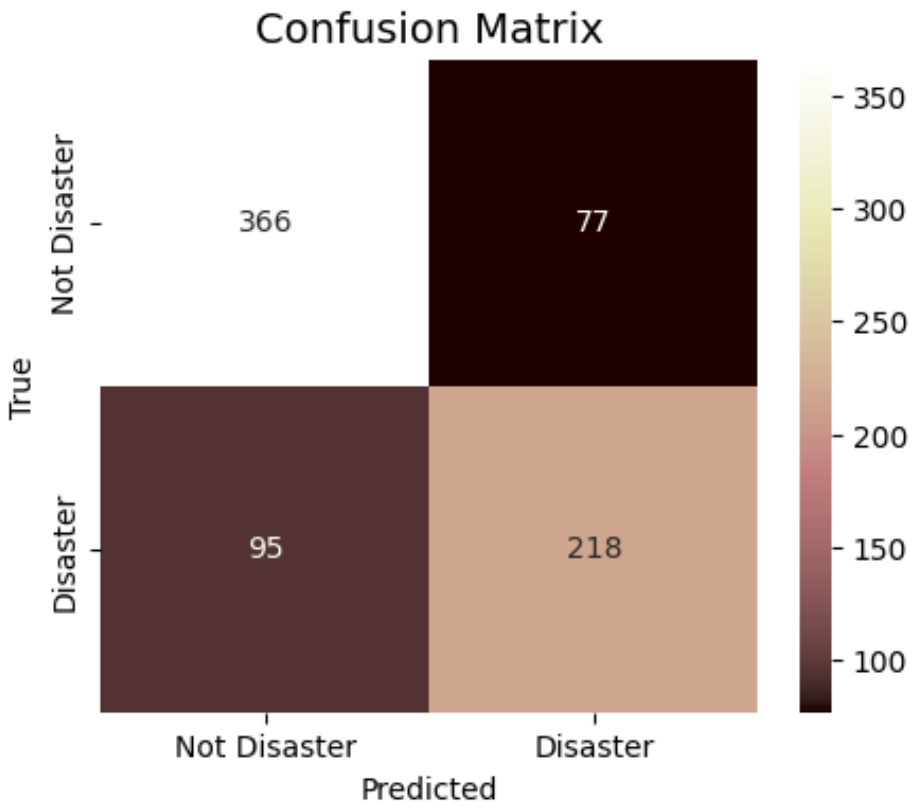


Figure: Confusion Matrix

Model Metrics

Accuracy: 0.772

Precision: 0.739

Recall: 0.696

F1 Score: 0.717

AUC: 0.761

	precision	recall	f1-score	support
0	0.793926	0.826185	0.809735	443.000000
1	0.738983	0.696486	0.717105	313.000000
accuracy	0.772487	0.772487	0.772487	0.772487
macro avg	0.766455	0.761335	0.763420	756.000000
weighted avg	0.771179	0.772487	0.771384	756.000000

Configuration 2

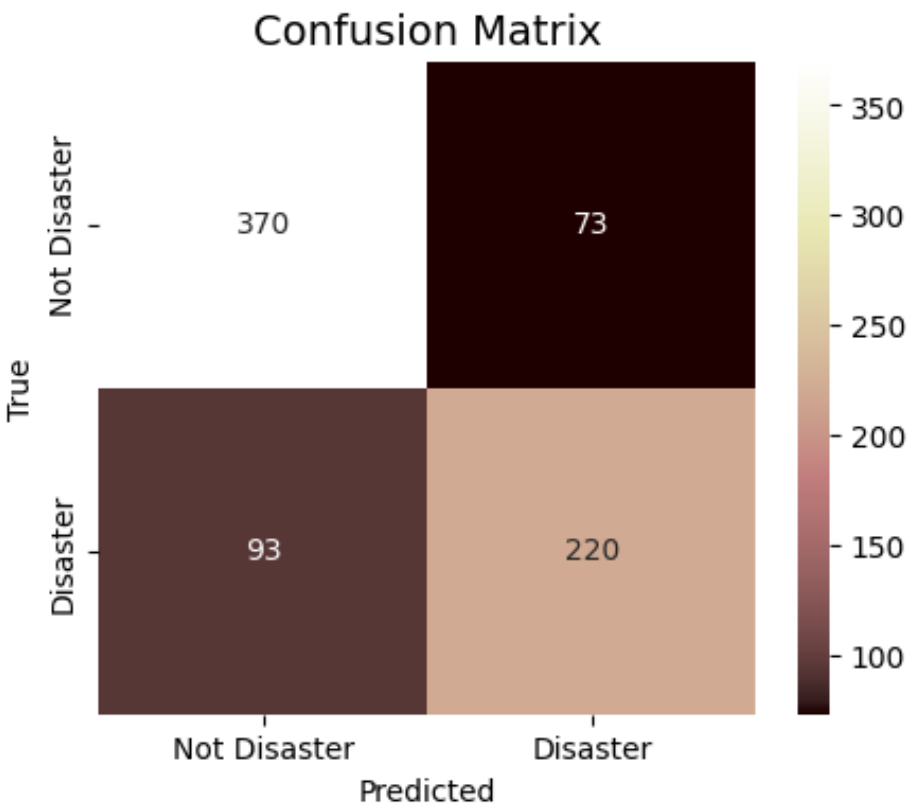


Figure: Confusion Matrix

Model Metrics

Accuracy: 0.780

Precision: 0.751

Recall: 0.703

F1 Score: 0.726

AUC: 0.769

	precision	recall	f1-score	support
0	0.799136	0.835214	0.816777	443.000000
1	0.750853	0.702875	0.726073	313.000000
accuracy	0.780423	0.780423	0.780423	0.780423
macro avg	0.774995	0.769045	0.771425	756.000000
weighted avg	0.779146	0.780423	0.779223	756.000000

Configuration 3

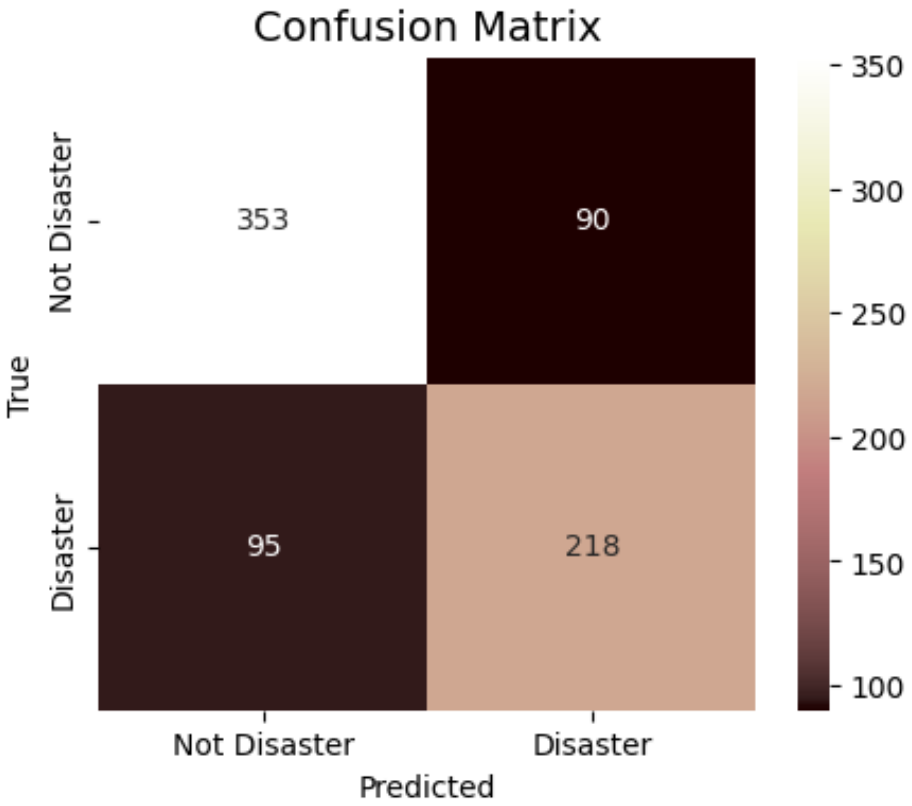


Figure: Confusion Matrix

	precision	recall	f1-score	support
0	0.787946	0.796840	0.792368	443.000000
1	0.707792	0.696486	0.702093	313.000000
accuracy	0.755291	0.755291	0.755291	0.755291
macro avg	0.747869	0.746663	0.747231	756.000000
weighted avg	0.754761	0.755291	0.754992	756.000000

Model Metrics

Accuracy: 0.755

Precision: 0.708

Recall: 0.696

F1 Score: 0.702

AUC: 0.747

Metrics Comparison

The bar plot shows the individual performance of each configuration across all evaluation metrics. This visualization allows a detailed comparison of Accuracy, Precision, Recall, F1 Score, and AUC for activation function.

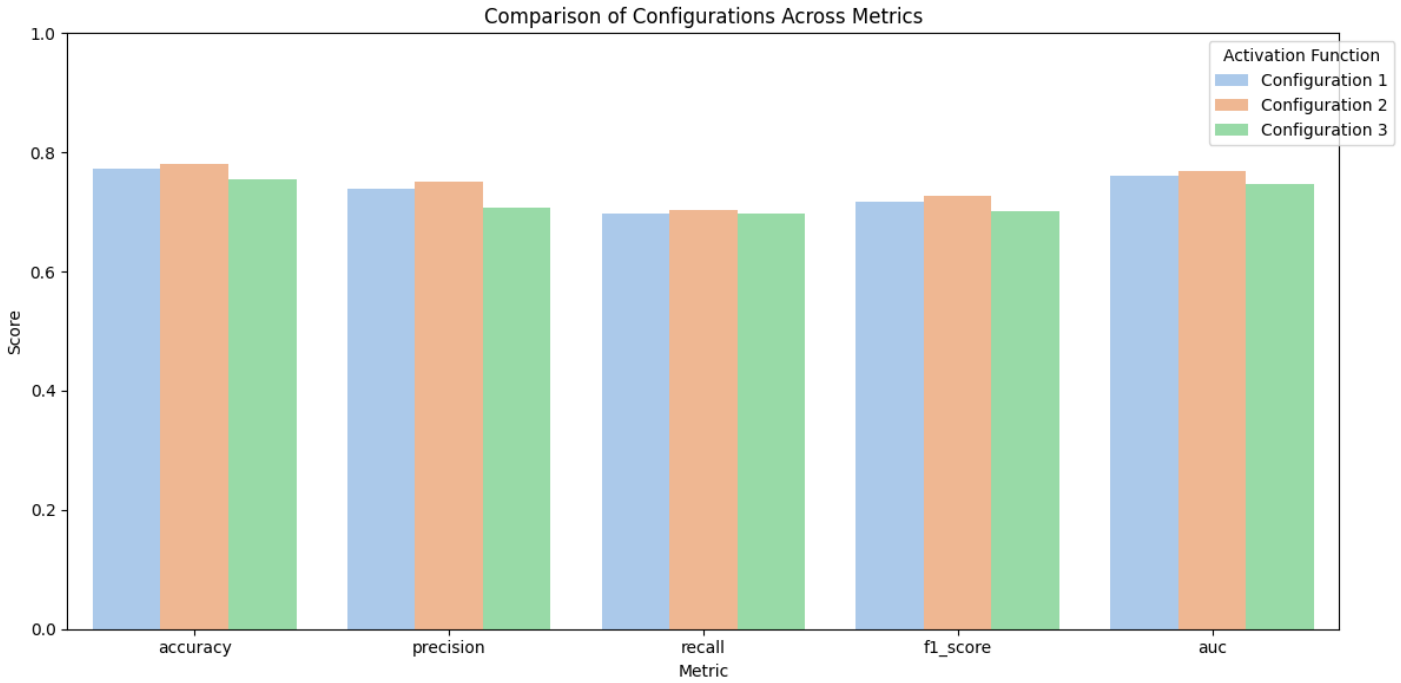
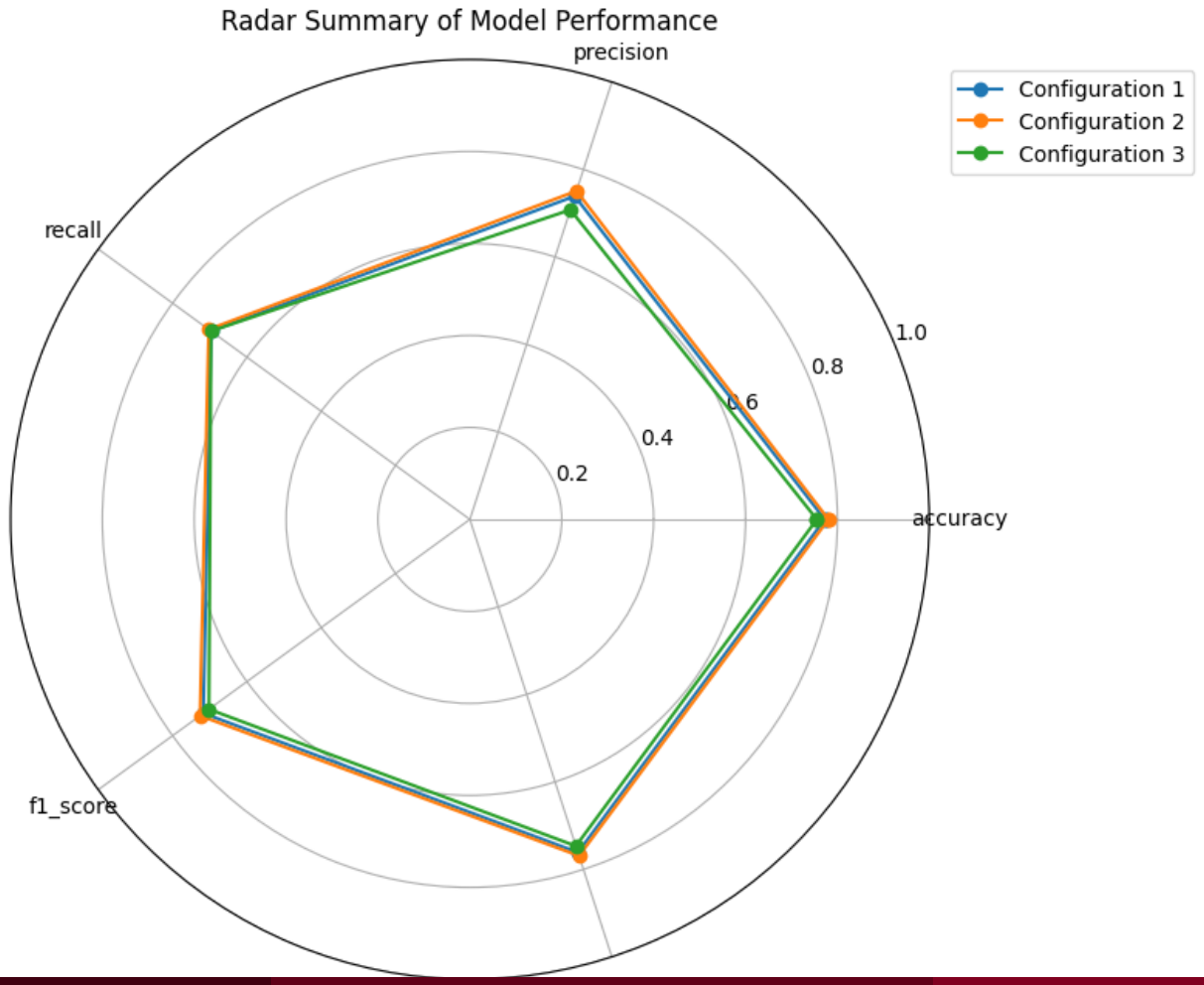


Figure: Performance Comparison Across Metrics and Configurations

Metrics Comparison

This representation highlights the balance and stability of each preprocessing configuration. Models with a more uniform and larger coverage area indicate more consistent performance across metrics.



ANALYSIS

Comparative Analysis

Configuration	Accuracy	Precision	Recall	F1-score	AUC
Configuration 1	0.7725	0.7390	0.6965	0.7171	0.7613
Configuration 2	0.7804	0.7509	0.7029	0.7261	0.7690
Configuration 3	0.7553	0.7078	0.6965	0.7021	0.7467

The results show only small differences in performance between the configurations.
However, Configuration 2 achieves the best overall performance. This configuration includes stopword removal, which likely improves the model because stopwords (e.g., "the", "and", "to") usually carry little semantic information and may introduce noise into the feature representation.

Technical Analysis

1. **What happens when stopwords are removed?**

   Removing stopwords slightly improves the model performance. These words usually carry little semantic information and may introduce noise into the TF–IDF representation.

2. **What happens when stopwords are kept?**

   The performance remains very similar. Although the difference is small in this dataset, keeping stopwords in other datasets could negatively affect the model due to the presence of many non-informative words.

3. **What is the impact of stemming or lemmatization?**

   In this experiment, applying lemmatization resulted in slightly worse performance. However, it can still be useful in some cases because it reduces vocabulary size and groups different forms of a word into a common base form.